Module 1: C# Interview Questions & Answers (8+ Years Experience).

Module 1 – C# Interview Questions & Answers (Part 1)

Question 1: What are the four pillars of Object-Oriented Programming (OOP)?

Answer

Object-Oriented Programming (OOP) is based on four fundamental principles:

1. Encapsulation

Encapsulation is the process of wrapping data (fields) and methods into a single unit (class) and restricting direct access to data.

Example:

```
public class Employee
{
   private decimal salary;

   public void SetSalary(decimal amount)
   {
      if (amount > 0)
         salary = amount;
   }

   public decimal GetSalary()
   {
      return salary;
   }
}
```

Advantages:

- Data hiding
- Better security
- Easy maintenance
- Controlled access

Interviewer Expectation:
Explain that encapsulation protects an object's internal state by exposing only necessary operations.

---

2. Abstraction

Abstraction hides implementation details and exposes only essential functionality.

Example:

```
public interface IPayment
{
   void Pay(decimal amount);
}


public class CreditCardPayment : IPayment
{
   public void Pay(decimal amount)
   {
      Console.WriteLine($"Paid {amount}");
   }
}
```

Advantages:

- Reduces complexity
- Improves maintainability
- Supports loose coupling

---

3. Inheritance

Inheritance allows one class to inherit properties and methods from another.

```
public class Animal
{
   public void Eat()
   {
      Console.WriteLine("Eating");
   }
}
```

```
public class Dog : Animal
{
   public void Bark()
   {
      Console.WriteLine("Barking");
   }
}
```

Advantages

- Code reuse
- Extensibility
- Easier maintenance

---

4. Polymorphism

Polymorphism allows the same method to behave differently depending on the object.

Compile-time (Method Overloading)

```
public class Calculator
{
   public int Add(int a, int b)
   {
      return a + b;
   }

   public double Add(double a, double b)
   {
      return a + b;
   }
}
```

Runtime (Method Overriding)

```
public class Animal
{
   public virtual void Speak()
```

```
    {
        Console.WriteLine("Animal");
    }
}

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Bark");
    }
}
```

Real-Time Example

Consider an online banking application:

- **Encapsulation:** Protect account balance with private fields.
- **Abstraction:** Expose only methods like Deposit() and Withdraw().
- **Inheritance:** SavingsAccount and CurrentAccount inherit from Account.
- **Polymorphism:** Each account type calculates interest differently.

Follow-up Questions

- What is data hiding?
- Why is abstraction important?
- Can C# support multiple inheritance?
- Difference between overloading and overriding?

Common Mistakes

❌ Saying encapsulation means only using private variables.

✓ Correct: Encapsulation combines data and behavior while controlling access.

Interview Tip

For 8+ years of experience, explain each concept with a practical example from your projects rather than just definitions.

Question 2: What is the difference between an Abstract Class and an Interface?

Answer

Both define contracts, but they serve different purposes.

Feature	Abstract Class	Interface
Multiple inheritance	No	Yes
Constructors	Supported	Not supported
Fields	Allowed	Not allowed (except constants)
Method implementation	Can have both implemented and abstract methods	Can define default implementations (C# 8+), primarily contracts
Access modifiers	Any	Members are public by default

Use an Abstract Class When

- You want to share common implementation.
- Classes have a common base.
- You need constructors or shared state.

Use an Interface When

- Unrelated classes need the same capability.
- You want loose coupling.
- You need dependency injection and testability.

Example

```
public abstract class Vehicle
{
   public abstract void Start();

   public void Stop()
   {
      Console.WriteLine("Stopped");
   }
}
```

```
public interface IGps
{
    void Navigate();
}

public class Car : Vehicle, IGps
{
    public override void Start()
    {
        Console.WriteLine("Car Started");
    }

    public void Navigate()
    {
        Console.WriteLine("GPS Navigation");
    }
}
```

Interviewer's Expectation

A senior developer should explain **when** to use an abstract class versus an interface, not just list differences.

Follow-up Questions

- Can an abstract class implement an interface?
- Can an interface inherit another interface?
- Why is dependency injection typically based on interfaces?

Common Mistake

❌ "Always use interfaces."

✓ Use interfaces for contracts and abstraction across unrelated types. Use abstract classes when you need shared implementation and state.

Next Module Content

I'll continue with:

- **Question 3:** ref, out, and in keywords
- **Question 4:** Value Types vs Reference Types
- **Question 5:** string vs StringBuilder
- **Question 6:** Boxing vs Unboxing
- **Question 7:** const, readonly, and static readonly
- **Question 8:** Delegates, Events, and Lambda Expressions
- **Question 9:** Extension Methods
- **Question 10:** Garbage Collection and IDisposable

This format will provide deep, interview-focused coverage for each topic before moving on to the next module.

Excellent. This is the final part of **Module 1 (C#)**. These topics are very common in interviews for **8+ years** because they test your understanding of advanced language features and memory management.

Module 1 – C# Interview Questions & Answers (Part 3)

Question 8: Explain Delegates, Events, and Lambda Expressions.

Answer

What is a Delegate?

A **delegate** is a type-safe function pointer. It holds a reference to one or more methods with the same signature.

Syntax

```
public delegate void Notify(string message);


public class NotificationService
{
    public void SendEmail(string message)
    {
        Console.WriteLine($"Email: {message}");
```

```
    }
}
```

```
NotificationService service = new NotificationService();
Notify notify = service.SendEmail;
notify("Order Placed Successfully");
```

Real-Time Use Cases

- Callback methods
- Event handling
- LINQ
- Multithreading (Task callbacks)
- Plugin architectures

Interview Follow-up

Q: Why use delegates instead of calling methods directly?

Answer:
Delegates enable **loose coupling**. The caller doesn't need to know which method will execute, making systems more flexible and extensible.

What is an Event?

An **event** is a mechanism built on delegates that allows a class to notify subscribers when something happens.

Example

```
public class Order
{
    public event Action<string> OrderPlaced;

    public void PlaceOrder()
    {
        OrderPlaced?.Invoke("Order placed successfully.");
    }
}
```

Real-Time Example

In an e-commerce application:

- Customer places an order.
- Email service sends a confirmation.
- SMS service sends a notification.
- Inventory service updates stock.

The Order class raises an event, and each service subscribes independently.

Interview Tip

Explain that **events implement the Publisher-Subscriber pattern**, reducing dependencies between components.

Lambda Expressions

A lambda expression is a concise way to define an anonymous function.

Example

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };


var evenNumbers = numbers.Where(x => x % 2 == 0);
```

Benefits

- Cleaner code
- Widely used with LINQ
- Supports functional programming

Interview Question

Q: Are lambda expressions delegates?

Answer:
A lambda expression can be compiled into a delegate or an expression tree, depending on the target type.

Question 9: What are Extension Methods?

Answer

Extension methods allow you to add new methods to an existing type **without modifying its source code or creating a derived class**.

Example

```
public static class StringExtensions
```

```
{
    public static bool IsEmail(this string value)
    {
        return value.Contains("@");
    }
}


string email = "user@example.com";
bool valid = email.IsEmail();
```

Real-Time Use Cases

- Utility/helper methods
- Common validation logic
- Formatting methods
- Reusable business logic

Advantages

- Improves readability
- Encourages reusable code
- Doesn't modify the original class

Interview Follow-up

Q: How do extension methods work internally?

Answer:
They are static methods in static classes. The this keyword on the first parameter allows them to be called using instance-method syntax.

Question 10: Explain Garbage Collection (GC), IDisposable, and using.

Answer

What is Garbage Collection?

The .NET **Garbage Collector (GC)** automatically reclaims memory occupied by objects that are no longer referenced.

Generations

Generation	Description
Gen 0	Short-lived objects
Gen 1	Objects that survive Gen 0 collection
Gen 2	Long-lived objects

Example

```
Employee emp = new Employee();
emp = null;

// Later, GC will reclaim the object if no references remain.
```

Interview Question

Q: Can we force garbage collection?

Answer:

```
GC.Collect();
```

While possible, **it's generally not recommended** because the GC is optimized to decide the best time to collect memory.

What is IDisposable?

IDisposable is used to release **unmanaged resources**, such as:

- File handles
- Database connections
- Network sockets
- Streams

Example

```
public class FileManager : IDisposable
{
   public void Dispose()
   {
      // Release unmanaged resources
   }
}
```

Why is it Important?

The Garbage Collector frees managed memory but **does not guarantee timely release of unmanaged resources**.

What is the using Statement?

The using statement ensures Dispose() is called automatically, even if an exception occurs.

Example

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
   connection.Open();
}
```

The connection is automatically disposed at the end of the block.

Common Interview Questions

Q: Difference between Finalizer and Dispose?

Dispose	Finalizer
Called explicitly or via using	Called by the Garbage Collector
Releases managed and unmanaged resources	Primarily releases unmanaged resources
Deterministic	Non-deterministic

Q: Does the Garbage Collector call Dispose() automatically?

Answer:
No. The GC reclaims memory, but it does **not** automatically call Dispose(). Developers should use using or explicitly call Dispose() for objects that implement IDisposable.

Module 1 Summary

You now have a solid foundation in C# with these 10 key interview topics:

1. OOP Principles
2. Abstract Class vs Interface
3. ref, out, and in
4. Value Types vs Reference Types
5. string vs StringBuilder

6. Boxing vs Unboxing
7. const, readonly, and static readonly
8. Delegates, Events, and Lambda Expressions
9. Extension Methods
10. Garbage Collection, IDisposable, and using

These questions are frequently asked in senior .NET interviews and often lead to deeper follow-up discussions.

Next Module

We'll move on to **Module 2: Advanced C#**, covering topics such as:

1. async/`await`
2. Task Parallel Library (TPL)
3. Multithreading vs Parallel Programming
4. Task.Run() vs Thread
5. Task.WhenAll() vs Task.WaitAll()
6. lock, Monitor, Mutex, and SemaphoreSlim
7. Cancellation Tokens
8. Thread Pool
9. Deadlocks and Race Conditions
10. IEnumerable, IQueryable, IAsyncEnumerable, and PLINQ`

These are among the most frequently asked topics for developers with 8+ years of experience.

Excellent. **Module 2 – Advanced C#** is one of the most important sections for senior .NET interviews. Interviewers often ask scenario-based questions rather than definitions.

We'll begin with the first two questions in detail.

Module 2 – Advanced C#

Question 1: Explain async and await in C#.

Answer

async and await simplify asynchronous programming. They allow long-running operations (database calls, HTTP requests, file I/O, etc.) to execute **without blocking the calling thread**.

Key point: async/`await` is about **non-blocking asynchronous operations**, not about creating new threads.

Why do we need async/`await`?

Imagine an ASP.NET Core Web API that calls:

- SQL Server
- Another REST API
- Azure Blob Storage

If each call blocks the thread:

Request

↓

Thread waits for SQL (2 sec)

↓

Thread waits for REST API (1 sec)

↓

Thread waits for Blob (2 sec)

The thread remains occupied while waiting.

With async/`await`:

Request

↓

SQL starts

Thread returned to thread pool

↓

SQL completes

Thread resumes execution

The thread can serve other requests while the operation is in progress, improving scalability.

Example

```
public async Task<string> GetEmployeeAsync()
{
   await Task.Delay(3000);   // Simulates I/O work
   return "Satheesh";
}
```

Calling it:

string employee = await GetEmployeeAsync();

What does async do?

async enables the use of await in a method.

public async Task SaveAsync()

{

}

Without async, you cannot use await.

What does await do?

await pauses the method until the awaited task completes **without blocking the thread**.

await repository.SaveEmployeeAsync();

While waiting:

- The current thread is released.
- Once the task finishes, execution resumes.

Real-Time Example

public async Task<IActionResult> GetEmployee(int id)

{

  var employee = await _repository.GetEmployeeAsync(id);

  return Ok(employee);

}

GetEmployeeAsync() performs a database call asynchronously, allowing the server to handle other requests during the wait.

Common Interview Questions

Q1: Does async create a new thread?

Answer:
No. async/`await` does not create a new thread. It enables non-blocking asynchronous execution. The underlying operation may use I/O completion ports or the thread pool as appropriate.

Q2: Can we have an async method without await?

Yes.

```
public async Task Test()
{
    Console.WriteLine("Hello");
}
```

However, the compiler issues a warning because the method runs synchronously.

Q3: Can we use await without async?

No.

Q4: Can the Main() method be async?

Yes.

```
static async Task Main()
{
}
```

Best Practices

✓ Prefer Task or Task<T> instead of void.

✓ Use async void only for event handlers.

✓ Avoid blocking async code with .Result or .Wait().

✓ Make the entire call chain asynchronous where possible.

Common Mistakes

❌ Calling:

```
var employee = repository.GetEmployeeAsync().Result;
```

This can cause deadlocks (especially in UI applications) and blocks the thread.

Interview Tip

A senior candidate should explain **how async improves scalability** by freeing threads during I/O waits, especially in ASP.NET Core applications.

Question 2: Explain the Task Parallel Library (TPL).

Answer

The **Task Parallel Library (TPL)** provides APIs for writing concurrent and parallel code using the Task type and related classes.

Namespace:

System.Threading.Tasks

Why use TPL?

Without TPL:

```
Thread thread = new Thread(DoWork);

thread.Start();
```

Problems:

- Manual thread management
- Poor scalability
- No built-in continuations
- More complex exception handling

With TPL:

```
Task.Run(() =>

{

   Console.WriteLine("Running...");

});
```

The runtime manages the thread pool efficiently.

Common TPL APIs

1. Task.Run()

Runs work on the thread pool.

```
await Task.Run(() =>

{

   Console.WriteLine("Processing...");

});
```

---

2. Parallel.For()

Executes loop iterations in parallel.

```
Parallel.For(1, 5, i =>
{
   Console.WriteLine(i);
});
```

Suitable for CPU-bound work.

---

3. Parallel.ForEach()

```
Parallel.ForEach(customers, customer =>
{
   Process(customer);
});
```

Useful for processing large collections concurrently.

---

4. Task.WhenAll()

Runs multiple asynchronous operations concurrently.

```
var employeeTask = GetEmployeeAsync();
var salaryTask = GetSalaryAsync();

await Task.WhenAll(employeeTask, salaryTask);
```

Both tasks execute at the same time, reducing overall latency.

---

5. Task.WhenAny()

Returns when the first task completes.

```
var completedTask = await Task.WhenAny(task1, task2);
```

Useful for timeouts and selecting the fastest response.

Real-Time Scenario

Suppose an API dashboard needs:

- Employee details
- Salary details
- Attendance
- Leave balance

Instead of:

Employee → Salary → Attendance → Leave

(total ≈ 8 seconds)

Run them concurrently:

```
await Task.WhenAll(
   GetEmployeeAsync(),
   GetSalaryAsync(),
   GetAttendanceAsync(),
   GetLeaveAsync());
```

If each call takes ~2 seconds, the total time is about **2 seconds**, not 8 (assuming they are independent).

Common Interview Questions

Q1: Difference between Task and Thread?

Task	**Thread**
Lightweight abstraction	OS thread
Uses thread pool by default	Dedicated thread
Better performance	More overhead
Supports continuations	Manual coordination

Q2: When should you use Parallel.For()?

For **CPU-bound** operations such as:

- Image processing
- Encryption
- Large numerical computations

Avoid it for I/O-bound operations like HTTP requests or database calls.

Q3: Is TPL suitable for database calls?

Typically, **no**. Database and HTTP operations are I/O-bound, so use asynchronous APIs (await) instead of Parallel.For.

Best Practices

✓ Use Task.WhenAll() for independent asynchronous operations.

✓ Use Parallel.For() only for CPU-bound workloads.

✓ Avoid creating threads manually unless there's a specific need.

✓ Handle exceptions properly when awaiting multiple tasks.

Interview Tip

A senior developer should clearly distinguish **parallelism** from **asynchrony**:

- **Asynchronous programming** improves responsiveness and scalability by not blocking while waiting.
- **Parallel programming** executes multiple pieces of work at the same time, typically to speed up CPU-intensive tasks.

Coming Next

In **Module 2 – Part 2**, we'll cover:

3. **Multithreading vs Parallel Programming**
4. **Task.Run() vs Thread**
5. **Task.WhenAll() vs Task.WaitAll()**

These questions are very common in interviews because they assess your understanding of concurrency and performance.

Great. These are among the **most frequently asked questions** in interviews for **8–15 years of .NET experience**. Interviewers often use them to evaluate your understanding of concurrency, performance, and application scalability.

Module 2 – Advanced C#

Part 2

Question 3: Explain the difference between Multithreading, Parallel Programming, and Asynchronous Programming.

This is one of the **top 10 C# interview questions**.

Answer

Although these concepts are related, they solve different problems.

Feature	**Multithreading**	**Parallel Programming**	**Asynchronous Programming**
Purpose	Run multiple threads	Execute multiple tasks simultaneously	Avoid blocking while waiting
Uses multiple threads	Yes	Yes (typically)	Not necessarily
Best for	Independent work	CPU-intensive work	I/O-bound operations
Improves	Responsiveness	Performance	Scalability

---

1. Multithreading

Multithreading means an application has multiple threads executing concurrently.

Example:

Thread thread1 = new Thread(PrintOrders);

Thread thread2 = new Thread(PrintInvoices);

thread1.Start();

thread2.Start();

Both threads execute independently.

Real-world Example

In an ATM system:

Thread 1

Withdraw Money

Thread 2

Print Receipt

Both operations can execute concurrently.

---

2. Parallel Programming

Parallel programming divides CPU-intensive work into multiple pieces that run simultaneously.

Example

```
Parallel.For(1,1000,i =>
{
   CalculateSalary(i);
});
```

Suitable for:

- Image Processing
- Video Rendering
- Machine Learning
- Encryption
- Large Calculations

---

3. Asynchronous Programming

Async programming prevents threads from waiting during I/O operations.

Example

```
await httpClient.GetAsync(url);
```

While waiting for the HTTP response,

The thread is returned to ThreadPool.

Real-Time Comparison

Banking Application

Multithreading

Thread 1 → Deposit

Thread 2 → Withdraw

Parallel Programming

Interest Calculation

↓

Split into

CPU1

CPU2

CPU3

CPU4

Async Programming

Call SQL Server

↓

Waiting...

↓

Thread returns to pool

↓

Continue after SQL responds

Interview Question

Which one is used most in ASP.NET Core?

Answer

Asynchronous Programming

because

- Database calls
- REST APIs
- Azure Storage
- Azure Blob
- Cosmos DB

are all I/O-bound operations.

Interview Tip

A senior developer should say:

We generally avoid creating threads manually in ASP.NET Core. Instead, we use asynchronous APIs (async/`await`) for I/O-bound work and parallel programming only for CPU-intensive scenarios.

Question 4

Difference between Task.Run() and Thread

Very frequently asked.

Answer

Thread	**Task.Run**
Creates OS Thread	Uses ThreadPool
Heavyweight	Lightweight
Manual Management	Automatic
Slower	Faster
Expensive	Efficient

Thread Example

```
Thread thread = new Thread(() =>
{
    Console.WriteLine("Running");
});
```

thread.Start();

Task.Run()

```
await Task.Run(() =>
{
   Console.WriteLine("Running");
});
```

Which is Better?

Almost always

Task.Run()

because

.NET ThreadPool manages

- Scheduling
- Reuse
- Performance

When should Thread be used?

Rare cases

Example

Long-running background service

Dedicated hardware communication

Real-time processing

Interview Question

Does Task create a Thread?

Answer

No.

Task is an abstraction.

The runtime decides

- ThreadPool Thread

or

- Existing Thread

or

- I/O Completion Port

depending on operation.

Real-world Example

Uploading 500 files.

Bad

Create

500 Threads

Good

500 Tasks

↓

ThreadPool

Much more scalable.

Question 5

Difference between Task.WhenAll() and Task.WaitAll()

Extremely important interview question.

Task.WhenAll()

Non-blocking.

await Task.WhenAll(task1, task2, task3);

Thread is NOT blocked.

Recommended for ASP.NET Core.

Task.WaitAll()

Blocking.

Task.WaitAll(task1, task2);

Current Thread waits.

Comparison

Task.WhenAll	Task.WaitAll
Async	Blocking
await	No await
Better scalability	Blocks thread
Preferred	Legacy

Example

Wrong

```
Task.WaitAll(GetEmployee(), GetSalary());
```

Correct

```
await Task.WhenAll(
   GetEmployee(),
   GetSalary());
```

Real-Time Scenario

Employee Dashboard

Need

- Employee
- Salary
- Leave
- Attendance

Bad

Employee

↓

Salary

↓

Attendance

↓

Leave

Total

8 Seconds

Good

Task.WhenAll()

↓

Employee

Salary

Attendance

Leave

Total

2 Seconds

Interview Question

Which one should be used inside Web API?

Answer

Always

Task.WhenAll()

because it doesn't block request threads.

Follow-up Questions

Can Task.WhenAll() fail?

Yes.

If one task fails,

all exceptions are aggregated.

Example

```
try
{
   await Task.WhenAll(tasks);
}
catch(Exception ex)
{
}
```

Does Task.WhenAll() guarantee execution order?

No.

Tasks complete whenever they finish.

Senior Interview Tip

For independent I/O operations, use Task.WhenAll() to maximize concurrency. Avoid Task.WaitAll() in ASP.NET Core because it blocks request threads and reduces scalability.

Common Follow-up Scenario

Interviewer: Your Web API calls three downstream services. Each takes about two seconds. How do you reduce response time?

Strong Answer:

If the calls are independent, I would start all three tasks without awaiting immediately, then await them together using Task.WhenAll(). This allows the services to execute concurrently, reducing the total response time to approximately the duration of the longest call instead of the sum of all three. I would also add proper timeout handling, cancellation support, structured logging, and exception handling to ensure resilience.

What's Next?

In **Part 3 of Module 2**, we'll cover some of the most challenging senior-level topics:

1. **lock vs Monitor vs Mutex vs SemaphoreSlim**
2. **Race Conditions and Deadlocks**
3. **CancellationToken**

4. **ThreadPool**
5. **Best practices for writing thread-safe code**

These topics are especially common in interviews for **10+ years of experience**, where interviewers expect you to explain not just *what* these constructs are, but *when* and *why* to use each one.

Excellent. This is the **most important section** of Advanced C#. These questions are asked in **Microsoft, Deloitte, EY, Accenture, Cognizant, Capgemini, TCS Digital, Infosys, Wipro, IBM** and many product companies because they test your understanding of **thread safety, synchronization, and concurrent programming**.

Module 2 – Advanced C#

Part 3

Question 6: Explain lock, Monitor, Mutex, and SemaphoreSlim.

Answer

When multiple threads access the same resource simultaneously, it can lead to:

- Data corruption
- Race conditions
- Deadlocks
- Inconsistent application state

Synchronization primitives help prevent these issues.

---

1. lock

The lock statement ensures that **only one thread** can execute a critical section of code at a time.

Example

```
private readonly object _lock = new object();

private int _counter = 0;


public void Increment()

{

   lock (_lock)

   {

      _counter++;
```

```
    }
}
```

What happens internally?

Thread 1 ----> lock acquired ----> Executes ----> Releases lock

Thread 2 ----> Waits

Thread 3 ----> Waits

Only one thread enters the critical section.

When should you use lock?

Use it when:

- Updating shared variables
- Writing to shared collections
- Updating cache
- File operations within the same process

Advantages

- Easy to use
- Fast
- Prevents race conditions
- Automatically releases the lock

Interview Question

Q: Can you lock on a string?

Answer:

No.

Avoid locking on:

- string
- this
- typeof(MyClass)

These objects may be shared unintentionally, leading to unexpected blocking.

Use:

private readonly object _lock = new object();

---

2. Monitor

Monitor provides the same synchronization mechanism as lock but offers additional control.

Example

```
private readonly object _lock = new object();


bool lockTaken = false;


try
{
   Monitor.Enter(_lock, ref lockTaken);


   Console.WriteLine("Critical Section");
}
finally
{
   if (lockTaken)
      Monitor.Exit(_lock);
}
```

Advantages over lock

- TryEnter()
- Timeout support
- Explicit lock release
- More control over synchronization

Interview Question

Difference between lock and Monitor

lock	Monitor
Simpler syntax	More control
Compiler translates it to Monitor	Explicit API
No timeout	Supports timeout
Easier to read	More flexible

Interview Tip

Explain that:

The lock keyword is syntactic sugar around Monitor.Enter() and Monitor.Exit().

---

3. Mutex

A Mutex is used for **cross-process synchronization**.

Example

```
Mutex mutex = new Mutex();


mutex.WaitOne();


try
{
   Console.WriteLine("Critical Section");
}
finally
{
   mutex.ReleaseMutex();
}
```

Real-Time Example

Only one instance of an application should run.

Application 1

✓ Running

Application 2

❌ Wait

This is achieved using a named Mutex.

Interview Question

Difference between lock and Mutex

lock	**Mutex**
Same process only	Cross-process
Faster	Slower
Lightweight	Heavyweight

---

4. SemaphoreSlim

A SemaphoreSlim allows **multiple threads** to access a resource, up to a specified limit.

Example

```
SemaphoreSlim semaphore = new SemaphoreSlim(3);


await semaphore.WaitAsync();


try
{
   Console.WriteLine("Processing...");
}
finally
{
   semaphore.Release();
}
```

Real-Time Example

Database connection pool

Maximum connections:

10

Requests:

Request 1 ✓

Request 2 ✓

...

Request 10 ✓

Request 11 waits

Interview Question

Difference between lock and SemaphoreSlim

lock	**SemaphoreSlim**
One thread	Multiple threads
Exclusive access	Limited concurrent access
Simple	Flexible

Senior Interview Question

Which synchronization mechanism would you use?

Updating Bank Balance

lock

API Rate Limiting

SemaphoreSlim

Single Application Instance

Mutex

Timeout Needed

Monitor

Question 7

Explain Race Condition

Answer

A race condition occurs when multiple threads access shared data simultaneously and the final result depends on the timing of execution.

Example

Counter = 0

Thread A

Read Counter = 0

Thread B

Read Counter = 0

Thread A

Counter = 1

Thread B

Counter = 1

Expected

2

Actual

1

This is a race condition.

C# Example

```
int counter = 0;
```

```
Parallel.For(0, 10000, i =>
{
   counter++;
});

Console.WriteLine(counter);
```

Output

9783

9954

9991

Different every execution.

Solution

```
lock (_lock)
{
   counter++;
}
```

or

```
Interlocked.Increment(ref counter);
```

Interview Question

Why is Interlocked faster?

Answer

It performs atomic operations without taking a traditional lock.

Question 8

Explain Deadlock

Very common interview question.

Answer

Deadlock occurs when two or more threads wait for each other indefinitely.

Example

Thread 1

Lock A

Waiting for B

Thread 2

Lock B

Waiting for A

Neither thread can continue.

C# Example

```
lock (lockA)
{
   lock (lockB)
   {
   }
}
```

Another Thread

```
lock (lockB)
{
   lock (lockA)
   {
   }
}
```

Both threads wait forever.

How to Prevent Deadlocks

✓ Acquire locks in a consistent order.

✓ Keep lock duration as short as possible.

✓ Avoid nested locks when possible.

✓ Use Monitor.TryEnter() with a timeout.

Interview Question

How do you debug deadlocks?

Answer

- Analyze thread dumps.
- Review synchronization order.
- Use Visual Studio Parallel Stacks/Threads windows.
- Use diagnostic tools (e.g., dotnet-trace, memory/thread dump analyzers).
- Enable structured logging around critical sections if needed.

Senior Interview Tip

If asked:

Have you faced a race condition?

A strong answer is:

Yes. In one project, multiple requests updated the same in-memory cache entry concurrently, causing inconsistent data. We resolved it by protecting the update with synchronization and reducing shared mutable state where possible.

Part 3 Summary

You have covered:

- lock
- Monitor
- Mutex
- SemaphoreSlim
- Race Conditions
- Deadlocks

- Thread-safe programming principles

These topics are frequently used to assess practical concurrency knowledge in senior .NET interviews.

Next (Module 2 – Part 4)

We'll complete Module 2 with:

1. **CancellationToken**
2. **ThreadPool**
3. **IEnumerable vs IQueryable vs IAsyncEnumerable**
4. **PLINQ**
5. **Advanced async/await interview scenarios**
6. **Top 25 rapid-fire Advanced C# interview questions**

These are excellent topics for demonstrating senior-level expertise in modern .NET.

Excellent choice. For **8+ years of experience**, it's much better to cover **all high-value interview questions** rather than stopping at exactly 10. This will result in a complete **Senior .NET Interview Handbook** of around **350–500 pages**, similar to what senior interviewers expect.

Complete Handbook Structure

We'll cover every major topic in depth:

Part 1: C#

- OOP
- Collections
- Generics
- Delegates
- Events
- Lambda Expressions
- LINQ
- Memory Management
- Exception Handling
- Reflection
- Attributes
- Extension Methods
- Nullable Types

- Dynamic
- Records
- Span<T>
- ReadOnlySpan<T>

~30 Questions

Part 2: Advanced C#

- Async/Await
- TPL
- Parallel Programming
- ThreadPool
- Synchronization
- Deadlocks
- Race Conditions
- Concurrent Collections
- Channels
- Cancellation Tokens
- ConfigureAwait
- Performance

~35 Questions

Part 3: .NET Core / ASP.NET Core

This will be one of the largest sections.

Topics include:

- .NET Framework vs .NET Core
- CLR
- Kestrel
- Generic Host
- Middleware
- Request Pipeline
- Dependency Injection

- Service Lifetimes
- Configuration
- Logging
- Options Pattern
- Filters
- Model Binding
- Validation
- Routing
- Authentication
- Authorization
- Identity
- JWT
- CORS
- HTTPS
- Caching
- Session
- Cookies
- Health Checks
- Background Services
- Hosted Services
- Minimal APIs
- gRPC
- SignalR
- Performance

~60 Questions

Part 4: REST API

- REST Principles
- Richardson Maturity Model
- HTTP Methods
- Status Codes

- Idempotency
- Pagination
- Filtering
- Sorting
- Versioning
- Swagger
- OpenAPI
- JWT
- OAuth
- API Gateway
- Rate Limiting
- ProblemDetails
- Global Exception Handling
- HATEOAS

~50 Questions

Part 5: Azure

- App Service
- Azure Functions
- Service Bus
- Event Grid
- Event Hub
- Storage Account
- Blob
- Queue
- Tables
- Cosmos DB
- SQL Database
- Key Vault
- Managed Identity
- App Configuration

- Application Insights
- Azure Monitor
- Networking
- Virtual Networks
- NSG
- Private Endpoints
- Azure Front Door

~70 Questions

Part 6: Azure DevOps

- Git
- Branching Strategy
- Pull Requests
- Pipelines
- YAML
- Build
- Release
- Deployment Slots
- Blue-Green Deployment
- Canary Deployment
- Infrastructure as Code
- Bicep
- ARM
- Terraform

~40 Questions

Part 7: SQL Server

- Indexes
- Execution Plans
- Stored Procedures
- Functions

- Views
- Triggers
- Transactions
- Isolation Levels
- Deadlocks
- Performance Tuning
- Partitioning
- Query Optimization

~45 Questions

Part 8: Microservices

- DDD
- CQRS
- Event Sourcing
- Saga Pattern
- Outbox Pattern
- API Gateway
- Service Discovery
- Resiliency
- Polly
- Circuit Breaker

~40 Questions

Part 9: Design Patterns

- SOLID
- Repository
- Unit of Work
- Factory
- Builder
- Singleton
- Strategy

- Decorator
- Adapter
- Observer
- Mediator

~35 Questions

Part 10: System Design

- Design URL Shortener
- Design Banking Application
- Design Insurance System
- Design Payment Gateway
- Design E-Commerce
- Caching Strategy
- Scalability
- Security
- Monitoring

~30 Questions

Part 11: Behavioral & Leadership

- Conflict Resolution
- Code Reviews
- Mentoring
- Estimation
- Stakeholder Communication
- Production Incidents
- Root Cause Analysis

~30 Questions

Total Coverage

- **Approximately 450–500 interview questions**
- **Detailed answers**

- **Real project scenarios**
- **Architecture diagrams (text-based where applicable)**
- **Best practices**
- **Common mistakes**
- **Follow-up questions**
- **Interviewer's expectations**
- **Production troubleshooting examples**
- **Performance optimization techniques**

Customized for Your Background

From our previous conversations, I know your focus areas include:

- ASP.NET Core & .NET
- REST APIs
- Azure PaaS
- Azure DevOps
- Bicep/ARM deployments
- Azure Storage
- Cosmos DB
- Managed Identity
- SQL Server
- CI/CD
- Enterprise application development

I'll tailor examples and scenarios around these technologies so they're directly relevant to the types of interviews you're preparing for.

Next Module

We'll start **Module 3: ASP.NET Core / .NET Core**, which is the most commonly tested area in senior .NET interviews. It will cover the request pipeline, dependency injection, middleware, authentication, configuration, logging, caching, performance, and many real-world scenarios in depth. This is typically the highest-weighted technical section in interviews for senior .NET developers.